LAPORAN UAS
COMPUTER VISION

Segmentasi Objek pada Weizmann Horse Database

Disusun Oleh Kelompok 13:

Gunawan Wibisono	187231041
Balya Ananta Wicaksana Annahid	187231033
Abyan Arif Izzat Satrio	187231034
Muhammad Herjuna Taraka	187231042

PROGRAM STUDI S-1 SISTEM INFORMASI
DEPARTEMEN MATEMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS AIRLANGGA

2026

DAFTAR ISI

BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan Penelitian 1
1.4 Manfaat Penelitian 2
BAB II LANDASAN TEORI 3
2.1 Citra Digital 3
2.2 Grayscaling 3
2.3 Image Enhancement 3
2.4 Histogram Equalization 3
2.5 Contrast Stretching 3
2.6 CLAHE 4
2.7 Median Filter 4
2.8 MSE dan PSNR 4
2.9 Segmentasi Objek 4
2.10 Deteksi Tepi Canny 4
2.11 Deteksi Tepi Laplacian 5
2.12 U-Net 5
2.13 DeepLabV3 5
2.14 IoU, Dice, dan Fungsi Loss 5
BAB III DATASET 6
BAB IV METODOLOGI 8
4.1 Alur Penelitian 8
4.2 Pra-pemrosesan Citra 9
4.3 Image Enhancement dan Median Filter 9
4.4 Evaluasi dan Penentuan Masukan Segmentasi 9
4.5 Segmentasi Metode Klasik (Canny dan Laplacian) 9
4.6 Segmentasi Metode Deep Learning (U-Net dan DeepLabV3) 9
4.7 Evaluasi Segmentasi dan Rancangan Eksperimen 10
4.8 Tools dan Library 10
BAB V PEMBAHASAN 11
5.1 Image Enhancement dan Pemilihan Masukan 11
5.2 Segmentasi Metode Klasik: Canny dan Laplacian 14

5.3 Segmentasi Deep Learning: U-Net dan DeepLabV3 17
5.4 Perbandingan 8 Output Segmentasi 23
BAB VI SARAN DAN KESIMPULAN 26
6.1 Kesimpulan 26
6.2 Saran 26
DAFTAR PUSTAKA 28
LAMPIRAN 29

DAFTAR GAMBAR

Gambar 3.1. Contoh citra dan mask ground truth (horse004, horse154, dan horse325)..... 6

Gambar 4.1. Alur penelitian..... 8

Gambar 5.1. Perbandingan MSE dan PSNR enam kandidat praproses (batang biru: terbaik) 14

Gambar 5.2. Contoh hasil: Contrast Stretching, keluaran Canny 21

Gambar 5.3. Contoh hasil: Contrast Stretching, keluaran Laplacian 22

Gambar 5.4. Contoh hasil: Contrast Stretching + Median, keluaran Canny 22

Gambar 5.5. Contoh hasil: Contrast Stretching + Median, keluaran Laplacian..... 23

DAFTAR TABEL

Tabel 3.1. Ringkasan dataset dan pembagian data 6

Tabel 4.1. Rancangan delapan output 10

Tabel 5.1. Evaluasi enam kandidat praproses (rata-rata 327 citra) 13

Tabel 5.2. IoU dan Dice metode klasik pada 65 citra uji 16

Tabel 5.3. IoU dan Dice deep learning pada 65 citra uji (60 epoch) 20

Tabel 5.4. IoU dan Dice kedelapan output pada 65 citra uji 23

Tabel 5.5. IoU metode klasik sendiri dibandingkan dengan lanjutan deep learning (65 citra uji) 24

Tabel 5.6. IoU pelatihan dari citra enhancement dibandingkan dengan dari peta tepi (65 citra uji) 24

Tabel 6.1. Ringkasan tahapan percobaan 26

Tabel L.1. Anggota Kelompok 13 29

BAB I
PENDAHULUAN

1.1 Latar Belakang

Segmentasi objek adalah proses memisahkan objek yang diamati dari latar belakangnya pada sebuah citra. Hasil segmentasi dapat dibandingkan secara langsung dengan mask ground truth, sehingga kualitas setiap metode dapat diukur secara kuantitatif. Pada tahap metode klasik, segmentasi kuda pada Weizmann Horse Database (Borenstein & Ullman, 2002) dikerjakan dengan metode klasik berbasis deteksi tepi, yaitu Canny (Canny, 1986) dan Laplacian (Marr & Hildreth, 1980). Tugas ini melanjutkan tahap metode klasik pada dataset yang sama dengan menambahkan dua metode berbasis deep learning, yaitu U-Net (Ronneberger et al., 2015) dan DeepLabV3 (Chen et al., 2017).

Kualitas citra masukan dapat memengaruhi hasil analisis, sehingga citra terlebih dahulu melalui tahap image enhancement. Tiga metode yang dibandingkan adalah Histogram Equalization, Contrast Stretching, dan CLAHE (Sharma & Kamra, 2023). Setiap metode dipakai dalam dua bentuk, yaitu tanpa filter dan diikuti median filter (Ahmed & Islam, 2023), sehingga terdapat enam kandidat yang dievaluasi dengan MSE dan PSNR (Aviantoro & Darnita, 2024) untuk menentukan kandidat terbaik. Kandidat terbaik kemudian menjadi masukan bagi Canny dan Laplacian, dan keluaran masing-masing dilanjutkan ke U-Net dan DeepLabV3 sehingga menghasilkan delapan output yang dievaluasi dengan IoU dan Dice.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah penelitian ini adalah sebagai berikut:

1. Bagaimana perbandingan enam kandidat praproses (Histogram Equalization, Contrast Stretching, dan CLAHE, masing-masing tanpa dan dengan median filter) berdasarkan MSE dan PSNR pada citra kuda, dan kandidat mana yang terbaik?
2. Bagaimana kinerja Canny dan Laplacian dalam memisahkan kuda dari latar belakang berdasarkan IoU dan Dice?
3. Bagaimana kinerja U-Net dan DeepLabV3 jika keluaran Canny dan Laplacian dijadikan masukan model?
4. Bagaimana perbandingan kedelapan output segmentasi, dan metode mana yang paling sesuai pada percobaan ini?

1.3 Tujuan Penelitian

Tujuan yang ingin dicapai dalam penelitian ini adalah:

1. Membandingkan enam kandidat enhancement dan filtering menggunakan MSE dan PSNR serta menentukan masukan segmentasi.
2. Menerapkan Canny dan Laplacian pada masukan terpilih dan mengevaluasinya dengan IoU dan Dice.
3. Melatih U-Net dan DeepLabV3 dengan keluaran Canny dan Laplacian sebagai masukan dan mengevaluasinya dengan IoU dan Dice.
4. Membandingkan seluruh output untuk menentukan metode segmentasi yang paling sesuai berdasarkan hasil percobaan.

1.4 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat sebagai berikut:

1. Bagi akademik, menjadi referensi penerapan image enhancement, deteksi tepi, dan deep learning untuk segmentasi objek pada citra kuda.
2. Bagi pengembangan teknologi, memberikan gambaran perbandingan metode klasik dan deep learning pada masukan yang sama beserta keterbatasan percobaannya.
3. Bagi penelitian selanjutnya, menjadi acuan untuk pengujian lebih lanjut, misalnya dengan beberapa pembagian data atau model berbobot pretrained.

BAB II
LANDASAN TEORI

2.1 Citra Digital

Citra digital adalah representasi visual dunia nyata dalam bentuk matriks dua dimensi yang terdiri atas piksel dengan nilai numerik tertentu. Komponen utamanya meliputi piksel sebagai unit dasar citra, resolusi yang menentukan tingkat detail, mode citra berupa grayscale, RGB, atau biner, serta format penyimpanan seperti JPEG, PNG, dan BMP (Dijaya & Setiawan, 2023).

2.2 Grayscaling

Grayscaling adalah proses mengubah citra berwarna menjadi citra skala abu-abu, dengan setiap piksel hanya memiliki satu nilai intensitas pada rentang 0 sampai 255. Pada penelitian ini konversi dilakukan dengan fungsi cv2.cvtColor (COLOR_BGR2GRAY) dari OpenCV, yang memberi bobot berbeda pada setiap kanal warna:

$$Gray = 0{,}299\,R + 0{,}587\,G + 0{,}114\,B$$

2.3 Image Enhancement

Image enhancement adalah pengolahan citra yang bertujuan memperbaiki kontras dan mengurangi noise agar kualitas citra meningkat sehingga hasilnya lebih sesuai untuk analisis berikutnya. Penelitian ini membandingkan tiga metode peningkatan kontras, yaitu Histogram Equalization, Contrast Stretching, dan CLAHE.

2.4 Histogram Equalization

Histogram Equalization (HE) memperbaiki kontras dengan mendistribusikan ulang intensitas piksel agar lebih seragam. HE memanfaatkan fungsi distribusi kumulatif dari histogram citra sehingga distribusi tingkat keabuan menjadi lebih lebar. Untuk citra 8-bit (L = 256), intensitas keluaran dihitung dengan:

$$s_k = (L - 1) \times \Sigma\,(n_j / N),\ \ j = 0, 1, \ldots, k$$

dengan s_k intensitas keluaran untuk tingkat keabuan k, n_j jumlah piksel dengan tingkat keabuan j, dan N jumlah seluruh piksel. Penelitian ini memakai fungsi cv2.equalizeHist.

2.5 Contrast Stretching

Contrast Stretching meregangkan rentang intensitas citra agar menempati rentang yang lebih lebar, dan pernah dibandingkan dengan metode lain menggunakan MSE dan PSNR pada citra buah (Aviantoro & Darnita, 2024). Pada penelitian ini batas bawah c dan batas atas d

diambil dari persentil ke-2 dan ke-98 intensitas citra, sedangkan rentang keluaran adalah a = 0 dan b = 255, dengan hasil dipotong ke rentang tersebut:

$$P_out = (P_in - c) \times (b - a) / (d - c) + a$$

2.6 CLAHE

Contrast Limited Adaptive Histogram Equalization (CLAHE) adalah pengembangan dari Adaptive Histogram Equalization yang bertujuan mengurangi penguatan noise berlebihan. CLAHE membatasi frekuensi histogram dengan parameter clip limit, lalu mendistribusikan kembali piksel yang terpotong ke bin lain. Dua parameter utamanya adalah clip limit dan ukuran tile yang menentukan area lokal pemerataan histogram (Sharma & Kamra, 2023). Penelitian ini memakai clip limit 2,0 dan tile 8 x 8.

2.7 Median Filter

Median filter adalah filter nonlinear yang mengurangi noise impulsif dengan mengganti nilai setiap piksel dengan median dari piksel di sekitarnya dalam jendela berukuran ganjil (Ahmed & Islam, 2023):

$$I_w(i, j) = median \{ I(i + r, j + s) \}, \ (r, s) \text{ di dalam jendela } w$$

Penelitian ini memakai jendela 5 x 5 (cv2.medianBlur).

2.8 MSE dan PSNR

Mean Squared Error (MSE) menghitung rata-rata kuadrat selisih nilai piksel antara citra asli dan citra hasil pemrosesan, sedangkan Peak Signal-to-Noise Ratio (PSNR) menyatakan rasio nilai maksimum piksel terhadap MSE dalam desibel (Aviantoro & Darnita, 2024). Semakin kecil MSE dan semakin besar PSNR, semakin kecil perubahan citra hasil terhadap citra asli.

$$MSE = (1/n) \Sigma (I_i - K_i)^2$$

$$PSNR = 10 \times log10 (255^2 / MSE)$$

dengan I_i nilai piksel citra asli, K_i nilai piksel citra hasil, dan n jumlah piksel. Pada program, nilai PSNR diberi batas 99 dB jika MSE sama dengan nol.

2.9 Segmentasi Objek

Segmentasi objek memisahkan wilayah objek dari latar belakang dan menghasilkan mask biner, yaitu piksel objek bernilai 1 dan piksel latar bernilai 0. Pada metode klasik, tepi hasil deteksi tepi dirapatkan dengan operasi morphological closing, kontur eksternal dicari, lalu

kontur yang memenuhi aturan luas dan posisi diisi penuh menjadi mask. Pada metode deep learning, model mempelajari pemetaan dari masukan ke mask ground truth.

2.10 Deteksi Tepi Canny

Detektor tepi Canny (Canny, 1986) menghasilkan citra tepi dari perubahan intensitas yang tajam. Pada OpenCV, fungsi cv2.Canny menerima dua ambang. Penelitian ini menetapkan ambang bawah 0,66 kali dan ambang atas 1,33 kali median intensitas citra, sehingga ambang menyesuaikan kecerahan masing-masing citra.

2.11 Deteksi Tepi Laplacian

Operator Laplacian merupakan turunan kedua yang merespons perubahan intensitas, dan penggabungannya dengan penghalusan Gaussian dikenal dalam teori deteksi tepi Marr dan Hildreth (1980). Pada penelitian ini citra dihaluskan dengan Gaussian (sigma 4), nilai mutlak respons Laplacian (kernel 3) dihitung, lalu piksel dengan respons di atas persentil ke-93 dijadikan tepi.

2.12 U-Net

U-Net adalah arsitektur jaringan konvolusi untuk segmentasi citra yang terdiri atas jalur penyempitan (encoder) untuk menangkap konteks dan jalur pelebaran (decoder) untuk melokalisasi objek, dengan skip connection yang menghubungkan keduanya (Ronneberger et al., 2015). Penelitian ini memakai versi kecil dengan empat tingkat encoder (16, 32, 64, dan 128 filter), bottleneck 256 filter, dan satu kanal keluaran sigmoid.

2.13 DeepLabV3

DeepLabV3 memakai atrous (dilated) convolution untuk memperluas bidang pandang tanpa mengurangi resolusi fitur, dan modul Atrous Spatial Pyramid Pooling (ASPP) untuk menangkap konteks pada beberapa skala (Chen et al., 2017). Penelitian ini memakai versi ringan dengan encoder ResNet kecil (output stride 8), ASPP dengan rate 6, 12, dan 18, dan upsampling bilinear. Model tidak memakai bobot pretrained.

2.14 IoU, Dice, dan Fungsi Loss

Kualitas segmentasi diukur dengan Intersection over Union (IoU) dan koefisien Dice, dengan P mask prediksi dan T mask ground truth. Keduanya bernilai 0 sampai 1, dan nilai lebih besar berarti lebih baik:

$$IoU = |P \cap T| / |P \cup T|$$

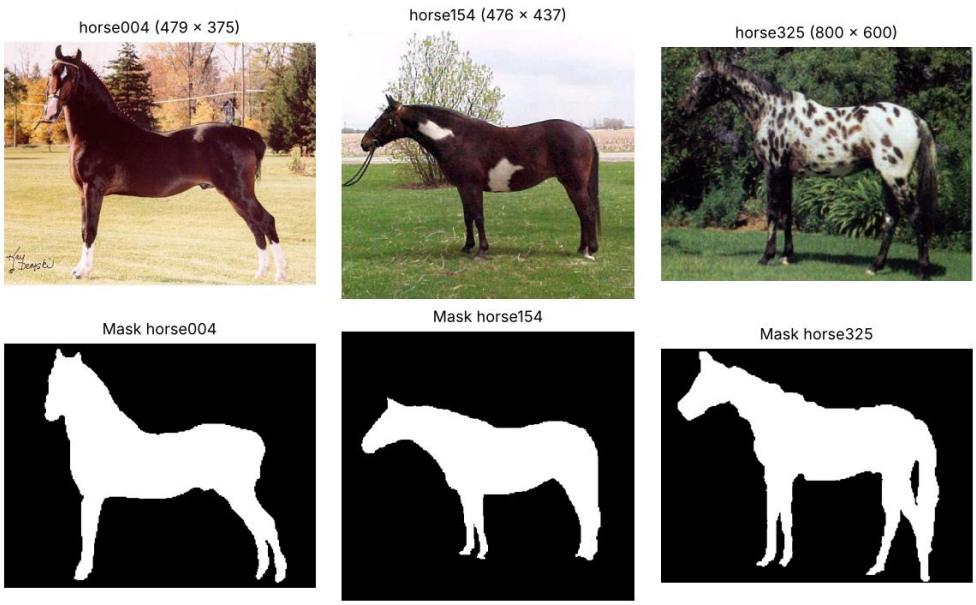
$$Dice = 2\,|P \cap T| \,/\, (|P| + |T|)$$

Pelatihan U-Net dan DeepLabV3 memakai loss berupa penjumlahan binary cross-entropy dan Dice loss, yaitu 1 dikurangi koefisien Dice yang dihitung pada keluaran sigmoid (dengan konstanta penghalus 1).

BAB III
DATASET

Dataset yang digunakan adalah Weizmann Horse Database (Borenstein & Ullman, 2002), yaitu kumpulan citra kuda beserta mask ground truth yang memisahkan kuda dari latar belakang. Pada penelitian ini terdapat 327 citra berformat PNG pada folder horse, dan setiap citra memiliki mask biner dengan nama berkas yang sama pada folder mask. Seluruh data disimpan pada folder weizmann_horse_db di dalam folder UAS_Kelompok.

Ukuran citra bervariasi. Tinggi citra berkisar antara 148 dan 960 piksel dan lebar antara 211 dan 1.280 piksel, dengan 259 kombinasi ukuran yang berbeda, dan ukuran yang paling banyak adalah 800 x 600 piksel (38 citra) disusul 640 x 480 piksel (23 citra). Nilai piksel mask hanya 0 dan 1. Luas area kuda pada mask berkisar antara 5,2% dan 43,1% dari luas citra, dengan rata-rata 25,2%.



Gambar 3.1. Contoh citra dan mask ground truth (horse004, horse154, dan horse325)

Tabel 3.1. Ringkasan dataset dan pembagian data

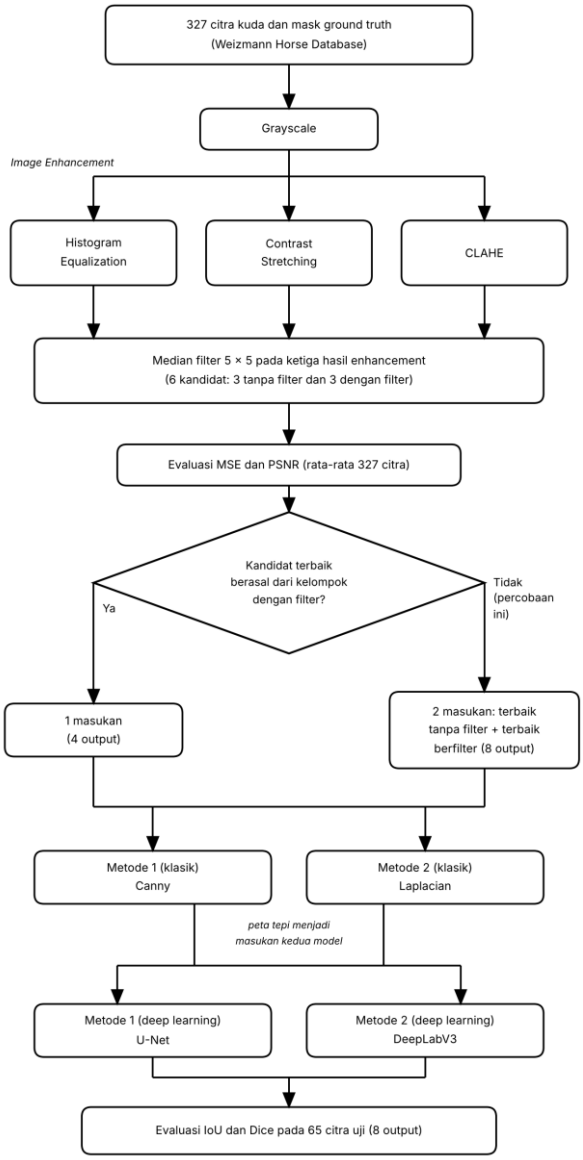
Keterangan	Nilai
Jumlah citra dan mask	327 berkas PNG
Rentang tinggi dan lebar citra	148 sampai 960 dan 211 sampai 1.280 piksel
Rata-rata luas area kuda pada mask	25,2% dari luas citra (rentang 5,2% sampai 43,1%)
Data latih	262 citra (di antaranya 15% sebagai validasi pada deep learning)
Data uji	65 citra (20%), pembagian dengan seed tetap 42
Ukuran masukan deep learning	128 x 128 piksel, satu kanal

Pembagian data dilakukan satu kali dengan seed tetap, yaitu 262 citra latih dan 65 citra uji (20%), dan pembagian yang sama dipakai oleh seluruh metode. Metode klasik tidak memerlukan pelatihan, tetapi tetap dievaluasi pada 65 citra uji yang sama agar dapat dibandingkan dengan U-Net dan DeepLabV3.

BAB IV
METODOLOGI

4.1 Alur Penelitian

Alur penelitian mengikuti ketentuan tugas yang melanjutkan tahap metode klasik. Citra diubah ke grayscale, lalu dilakukan tiga enhancement dan median filter pada ketiganya sehingga terbentuk enam kandidat. Setiap kandidat dievaluasi dengan MSE dan PSNR. Jika kandidat terbaik berasal dari kelompok yang tidak difilter, kandidat terbaik dari kelompok berfilter ikut dipakai sehingga terdapat dua masukan, dan hasil akhirnya berjumlah delapan output. Setiap masukan diproses oleh Metode 1 dan Metode 2 (Canny dan Laplacian), dan keluaran masing-masing dilanjutkan ke model deep learning (U-Net dan DeepLabV3).



Gambar 4.1. Alur penelitian

4.2 Pra-pemrosesan Citra

Citra berwarna dibaca dengan OpenCV dan diubah ke grayscale. Mask ground truth dibaca sebagai citra biner. Untuk deep learning, citra dan peta tepi diperkecil menjadi 128 x 128 piksel, mask diperkecil dengan interpolasi nearest neighbor agar tetap biner, sedangkan citra dan peta tepi diperkecil dengan interpolasi INTER_AREA. Metode klasik dijalankan pada resolusi asli.

4.3 Image Enhancement dan Median Filter

Tiga metode enhancement yang digunakan adalah Histogram Equalization, Contrast Stretching (persentil ke-2 dan ke-98), dan CLAHE (clip limit 2,0, tile 8 x 8). Median filter berukuran 5 x 5 diterapkan pada hasil ketiganya, sehingga terdapat enam kandidat, yaitu tiga tanpa filter dan tiga dengan filter. Filter yang dipilih adalah median filter, dan filter yang sama dipakai secara konsisten pada ketiga metode.

4.4 Evaluasi dan Penentuan Masukan Segmentasi

Setiap kandidat dievaluasi dengan MSE dan PSNR terhadap citra grayscale asli pada seluruh 327 citra, kemudian dirata-ratakan. Kandidat dengan rata-rata PSNR tertinggi menjadi kandidat terbaik. Jika kandidat terbaik tidak berasal dari kelompok berfilter, kandidat dengan PSNR tertinggi pada kelompok berfilter dipilih sebagai masukan kedua.

4.5 Segmentasi Metode Klasik (Canny dan Laplacian)

Canny dan Laplacian dijalankan pada setiap masukan sesuai parameter pada Subbab 2.10 dan 2.11. Untuk metode klasik, tepi dirapatkan dengan morphological closing (kernel 7 untuk Canny dan 15 untuk Laplacian), lalu kontur eksternal dicari. Kontur dipertahankan jika luasnya antara 2% dan 70% luas citra dan titik pusatnya berada pada 20% sampai 80% lebar dan tinggi citra, dan kontur terluas yang lolos diisi menjadi mask. Pada alur deep learning, peta tepi keluaran Canny dan Laplacian itu sendiri (bukan mask hasil konversi) diperkecil ke 128 x 128 dan dijadikan masukan deep learning.

4.6 Segmentasi Metode Deep Learning (U-Net dan DeepLabV3)

U-Net dan DeepLabV3 dilatih dari nol tanpa bobot pretrained dengan peta tepi satu kanal berukuran 128 x 128 sebagai masukan dan mask ground truth sebagai target. Loss yang dipakai adalah binary cross-entropy ditambah Dice loss, optimizer Adam dengan laju belajar awal 0,001 (dikurangi setengahnya jika tidak membaik selama 5 epoch), ukuran batch 16, dan 60 epoch. Augmentasi berupa cermin horizontal, geser 10%, zoom 15%, dan rotasi kecil

diterapkan pada peta tepi dan mask secara bersamaan. Sebanyak 15% data latih dipakai sebagai validasi, dan bobot dengan val_loss terbaik setelah epoch ke-10 disimpan.

4.7 Evaluasi Segmentasi dan Rancangan Eksperimen

Seluruh output dievaluasi dengan IoU dan Dice rata-rata pada 65 citra uji, dan jumlah citra uji dengan IoU minimal 0,5 juga dicatat. Rancangan eksperimen menghasilkan delapan output sebagaimana pada Tabel 4.1.

Tabel 4.1. Rancangan delapan output

Output	Masukan	Metode Klasik	Metode Deep Learning
1	Contrast Stretching	Canny	U-Net
2	Contrast Stretching	Canny	DeepLabV3
3	Contrast Stretching	Laplacian	U-Net
4	Contrast Stretching	Laplacian	DeepLabV3
5	Contrast Stretching + Median	Canny	U-Net
6	Contrast Stretching + Median	Canny	DeepLabV3
7	Contrast Stretching + Median	Laplacian	U-Net
8	Contrast Stretching + Median	Laplacian	DeepLabV3

4.8 Tools dan Library

Seluruh program ditulis dengan bahasa Python. Library yang digunakan sesuai berkas requirements.txt, yaitu NumPy, OpenCV (opencv-python), Matplotlib, dan TensorFlow (Keras). Program terdiri dari dua berkas, yaitu uas_tahap1.py (enhancement, evaluasi enam kandidat, Canny, Laplacian, dan ekspor data) dan uas_dl.py (pembuatan peta tepi Canny dan Laplacian sebagai masukan, serta pelatihan dan evaluasi U-Net dan DeepLabV3).

Cara menjalankan program:

```
pip install -r requirements.txt
python uas_tahap1.py --data weizmann_horse_db
python uas_dl.py --epochs 60
```

Interpretasi: Perintah pertama memasang library, perintah kedua menjalankan enhancement, evaluasi, Canny, Laplacian, dan ekspor data, perintah ketiga mengekspor peta tepi sebagai masukan deep learning, dan perintah keempat melatih U-Net dan DeepLabV3 selama 60 epoch. Hasil disimpan pada folder hasil.

BAB V
PEMBAHASAN

Bab ini menyajikan pelaksanaan percobaan secara berurutan. Setiap langkah ditampilkan dengan potongan kode, output yang dihasilkan, dan interpretasinya.

5.1 Image Enhancement dan Pemilihan Masukan

Subbab ini membahas parameter program, definisi enam kandidat, pembagian data latih dan uji, serta evaluasi dan penentuan masukan segmentasi.

5.1.1 Parameter Program

Program membaca beberapa parameter lewat argparse, yaitu lokasi data (--data), folder keluaran (--out), jenis filter (--filter, median atau gaussian), ukuran kernel filter (--k), dan ukuran citra untuk deep learning (--size). Nilai bawaannya adalah filter median, kernel 5, dan ukuran citra 128 x 128. Pada percobaan ini filter yang digunakan adalah median dengan kernel 5.

```
ap = argparse.ArgumentParser()
ap.add_argument("--data", default="../UTS_Kelompok/data", help="folder berisi horse/ dan
mask/")
ap.add_argument("--out", default="hasil")
ap.add_argument("--filter", default="median", choices=["median", "gaussian"])
ap.add_argument("--k", type=int, default=5)
ap.add_argument("--size", type=int, default=128, help="ukuran citra untuk deep learning")
args = ap.parse_args()
os.makedirs(args.out, exist_ok=True)
```

Interpretasi: Sel ini hanya membaca parameter dan membuat folder keluaran sehingga tidak menampilkan output. Berdasarkan ringkasan_tahap1.txt, percobaan dijalankan dengan filter median (kernel 5) pada 327 citra.

5.1.2 Fungsi Enhancement dan Enam Kandidat

Tiga metode enhancement yang dipakai adalah Histogram Equalization (cv2.equalizeHist), Contrast Stretching (fungsi buatan sendiri yang meregangkan intensitas antara persentil ke-2 dan ke-98), dan CLAHE (clipLimit 2,0 dan tileGridSize 8 x 8). Setiap metode dipakai dalam dua bentuk, yaitu tanpa filter dan diikuti filter median, sehingga terbentuk enam kandidat.

```
def contrast_stretch(g, lo=2, hi=98):
    a, b = np.percentile(g, (lo, hi))
    return np.clip((g.astype(np.float32) - a) * 255 / max(b - a, 1), 0,
255).astype(np.uint8)



ENH = {"Histogram Equalization": cv2.equalizeHist, "Contrast Stretching": contrast_stretch,
       "CLAHE": cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply}
filt = (lambda g: cv2.medianBlur(g, args.k)) if args.filter == "median" else (lambda g:
cv2.GaussianBlur(g, (args.k, args.k), 0))
FN = args.filter.capitalize()
```

```
# 6 kandidat: 3 enhancement saja + 3 enhancement dengan filter yang sama
CAND = {}
for n, f in ENH.items():
    CAND[n] = f
    CAND["%s + %s" % (n, FN)] = (lambda f: lambda g: filt(f(g)))(f)
```

Interpretasi: Kamus CAND berisi enam kandidat: Histogram Equalization, Histogram Equalization + Median, Contrast Stretching, Contrast Stretching + Median, CLAHE, dan CLAHE + Median. Contrast Stretching memotong 2% intensitas terendah dan 2% tertinggi sebelum meregangkan sisanya ke rentang 0 sampai 255, sehingga nilai ekstrem tidak ikut menentukan rentang peregangan.

5.1.3 Pembagian Data Latih dan Uji

Pembagian data dilakukan satu kali dengan RandomState(42), yaitu 20% citra (65 dari 327) menjadi data uji dan sisanya (262) menjadi data latih. Indeks yang sama dipakai oleh Canny, Laplacian, U-Net, dan DeepLabV3 sehingga seluruh metode dievaluasi pada citra uji yang sama.

```
# ---- 2) split data (sama untuk Canny/Laplacian dan deep learning) ----
rs = np.random.RandomState(42)
perm = rs.permutation(len(files))
n_te = int(round(0.2 * len(files)))
test_idx = np.sort(perm[:n_te])
is_test = np.zeros(len(files), bool); is_test[test_idx] = True
```

Interpretasi: Hasil pembagian disimpan pada berkas data_dl.npz bersama mask berukuran 128 x 128, dan dipakai ulang oleh uas_dl.py. Pada pelatihan deep learning, 15% dari data latih dipisahkan lagi sebagai data validasi untuk memilih bobot terbaik, sehingga data uji tidak dipakai selama pelatihan.

5.1.4 Perhitungan MSE dan PSNR

Setiap kandidat dievaluasi dengan MSE (Mean Squared Error) dan PSNR (Peak Signal-to-Noise Ratio) terhadap citra grayscale asli. MSE adalah rata-rata kuadrat selisih nilai piksel, sedangkan PSNR dihitung dengan rumus 10 log10(255^2 / MSE), dan bernilai 99 dB jika MSE sama dengan nol. MSE yang lebih kecil dan PSNR yang lebih besar menunjukkan perubahan terhadap citra asli yang lebih kecil.

```
def mse_psnr(ref, img):
    mse = float(np.mean((ref.astype(np.float64) - img.astype(np.float64)) ** 2))
    return mse, (99.0 if mse == 0 else 10 * np.log10(255 ** 2 / mse))


# ---- 1) evaluasi 6 kandidat (MSE & PSNR terhadap grayscale asli) ----
rows = []
for p in files:
    g = gray(p)
    for n, f in CAND.items():
        mse, psnr = mse_psnr(g, f(g))
        rows.append([name(p), n, round(mse, 4), round(psnr, 4)])
with open(os.path.join(args.out, "evaluasi_6_kandidat.csv"), "w", newline="") as fh:
```

```
    w = csv.writer(fh); w.writerow(["citra", "metode", "MSE", "PSNR_dB"]);
w.writerows(rows)
avg = {n: (np.mean([r[2] for r in rows if r[1] == n]), np.mean([r[3] for r in rows if r[1]
== n])) for n in CAND}
best = max(avg, key=lambda n: avg[n][1])
filtered = [n for n in CAND if n.endswith(" + " + FN)]
best_filt = max(filtered, key=lambda n: avg[n][1])
inputs = [best] if best in filtered else [best, best_filt]
```

Interpretasi: Evaluasi dilakukan pada 327 citra untuk 6 kandidat sehingga menghasilkan 1.962 baris pada berkas evaluasi_6_kandidat.csv. Kandidat terbaik dipilih dari rata-rata PSNR tertinggi, lalu dicek apakah kandidat itu berasal dari kelompok yang difilter untuk menentukan jumlah masukan segmentasi.

5.1.5 Hasil Evaluasi Enam Kandidat

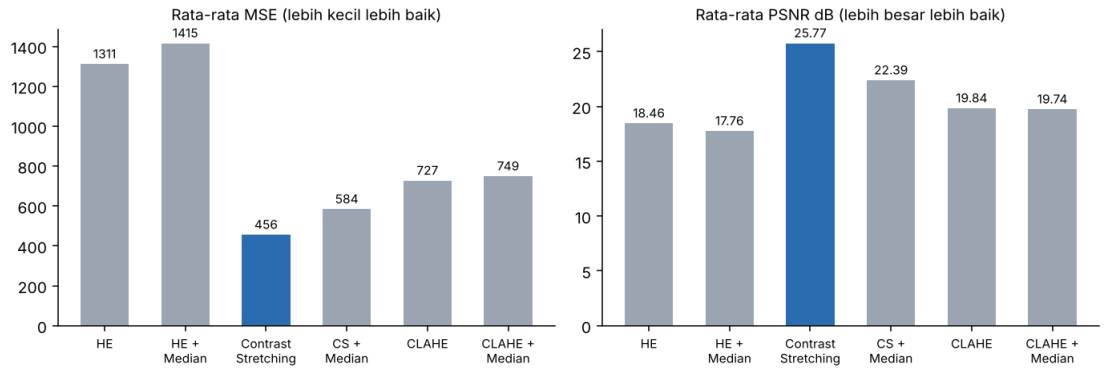
Output:

```
Filter: median (kernel 5) | jumlah citra: 327

Kandidat                              Rata2 MSE   Rata2 PSNR
Histogram Equalization                  1310.75     18.46 dB
Histogram Equalization + Median         1415.37     17.76 dB
Contrast Stretching                      455.56     25.77 dB
Contrast Stretching + Median             583.68     22.39 dB
CLAHE                                    726.92     19.84 dB
CLAHE + Median                           748.59     19.74 dB
```

Tabel 5.1. Evaluasi enam kandidat praproses (rata-rata 327 citra)

Kandidat	Rata-rata MSE	Rata-rata PSNR (dB)
Histogram Equalization	1310,75	18,46
Histogram Equalization + Median	1415,37	17,76
Contrast Stretching	**455,56**	**25,77**
Contrast Stretching + Median	583,68	22,39
CLAHE	726,92	19,84
CLAHE + Median	748,59	19,74



Gambar 5.1. Perbandingan MSE dan PSNR enam kandidat praproses (batang biru: terbaik)

Interpretasi: Contrast Stretching tanpa filter memberi PSNR tertinggi (25,77 dB) dan MSE terendah (455,56). Filter median tidak meningkatkan hasil pada ketiga metode: PSNR turun 0,70 dB pada Histogram Equalization (18,46 menjadi 17,76), 3,38 dB pada Contrast Stretching (25,77 menjadi 22,39), dan 0,10 dB pada CLAHE (19,84 menjadi 19,74). Perlu dicatat bahwa MSE dan PSNR di sini dihitung terhadap citra asli, sehingga nilai tinggi berarti perubahan dari citra asli kecil, bukan otomatis berarti citra lebih mudah disegmentasi.

5.1.6 Penentuan Masukan Segmentasi

Ketentuan tugas menyatakan bahwa jika kandidat terbaik bukan berasal dari kelompok yang difilter, maka kandidat terbaik dari kelompok yang difilter ikut dipakai sebagai masukan kedua. Karena Contrast Stretching tidak memakai filter, kandidat terbaik dari kelompok berfilter, yaitu Contrast Stretching + Median (PSNR 22,39 dB), dipakai sebagai masukan kedua.

Output:

```
Terbaik dari 6 (PSNR tertinggi): Contrast Stretching
Berasal dari kelompok filter: TIDAK -> 2 input (Contrast Stretching dan Contrast
Stretching + Median), 8 output
Input yang dipakai: Contrast Stretching; Contrast Stretching + Median
```

Interpretasi: Terdapat dua masukan, yaitu Contrast Stretching dan Contrast Stretching + Median. Setiap masukan diproses oleh Canny dan Laplacian (Metode 1 dan 2), lalu keluaran masing-masing dilanjutkan ke U-Net dan DeepLabV3 (metode deep learning), sehingga setiap masukan menghasilkan 4 output dan seluruhnya 8 output segmentasi.

5.2 Segmentasi Metode Klasik: Canny dan Laplacian

5.2.1 Deteksi Tepi Canny dan Laplacian

Canny dijalankan dengan ambang bawah 0,66 kali dan ambang atas 1,33 kali median intensitas citra, sehingga ambang menyesuaikan kecerahan masing-masing citra. Laplacian dijalankan setelah Gaussian blur (sigma 4), nilai mutlak responsnya dihitung dengan kernel 3, lalu piksel dengan respons di atas persentil ke-93 dijadikan tepi.

```
def canny(g):
    m = np.median(g)
    return cv2.Canny(g, 0.66 * m, 1.33 * m)



def laplacian(g):
    g = cv2.GaussianBlur(g, (0, 0), 4)
    l = np.abs(cv2.Laplacian(g, cv2.CV_64F, ksize=3))
    return ((l > np.percentile(l, 93)) * 255).astype(np.uint8)
```

Interpretasi: Kedua fungsi menghasilkan citra tepi biner, bukan mask objek. Pada metode klasik tepi ini diubah menjadi area objek pada subbab berikutnya, sedangkan pada alur deep learning peta tepi ini langsung menjadi masukan U-Net dan DeepLabV3 (Subbab 5.2.5).

5.2.2 Konversi Tepi menjadi Mask

Tepi dirapatkan dengan operasi morphological closing (kernel 7 untuk Canny dan 15 untuk Laplacian), lalu kontur eksternal dicari dengan findContours. Kontur dipertahankan jika luasnya antara 2% dan 70% luas citra dan titik pusatnya berada pada 20% sampai 80% lebar dan tinggi citra. Kontur terluas di antara kontur yang lolos diisi penuh sebagai mask prediksi.

```
CLOSE_K = {"Canny": 7, "Laplacian": 15}



def to_mask(edge, k):
    e = cv2.morphologyEx(edge, cv2.MORPH_CLOSE, np.ones((k, k), np.uint8))
    cnt,   = cv2.findContours(e, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    H, W = e.shape
    ok = []
    for c in cnt:
        a = cv2.contourArea(c)
        M = cv2.moments(c)
        cx, cy = M["m10"] / max(M["m00"], 1), M["m01"] / max(M["m00"], 1)
        if 0.02 * H * W <= a <= 0.7 * H * W and 0.2 * W < cx < 0.8 * W and 0.2 * H < cy <
0.8 * H:
            ok.append(c)
    m = np.zeros_like(e)
    if ok:
        cv2.drawContours(m, [max(ok, key=cv2.contourArea)], -1, 255, cv2.FILLED)
    return m
```

Interpretasi: Aturan ini sederhana dan tidak mempelajari bentuk kuda. Jika tidak ada kontur yang lolos, mask yang dihasilkan kosong dan IoU citra tersebut bernilai 0. Pada berkas segmentasi_klasik.csv, nilai IoU minimum pada keempat kombinasi masukan dan metode adalah 0,0, sehingga ada citra yang sama sekali tidak beririsan dengan ground truth.

5.2.3 Metrik IoU dan Dice

Mask prediksi dibandingkan dengan ground truth menggunakan IoU (Intersection over Union), yaitu luas irisan dibagi luas gabungan, dan Dice, yaitu dua kali luas irisan dibagi jumlah luas kedua mask. Keduanya bernilai 0 sampai 1, dan nilai lebih besar berarti lebih baik.

```
def iou_dice(pred, gt):
    p, t = pred > 0, gt > 0
    inter = (p & t).sum()
    return inter / max((p | t).sum(), 1), 2 * inter / max(p.sum() + t.sum(), 1)
```

Interpretasi: IoU selalu lebih kecil atau sama dengan Dice pada pasangan mask yang sama, sehingga nilai Dice pada tabel hasil lebih besar daripada IoU (misalnya 0,2250 berbanding 0,1705 untuk Canny pada masukan Contrast Stretching).

5.2.4 Hasil Canny dan Laplacian

Tabel 5.2. IoU dan Dice metode klasik pada 65 citra uji

Masukan	Metode	IoU	Dice	Citra IoU ≥ 0,5
Contrast Stretching	Canny	0,1705	0,2250	10 dari 65
Contrast Stretching	Laplacian	0,2567	0,3742	10 dari 65
Contrast Stretching + Median	Canny	0,2282	0,3077	13 dari 65
Contrast Stretching + Median	Laplacian	0,2243	0,3378	6 dari 65

Output:

```
Masukan                       Metode     Subset   Rata2 IoU  Rata2 Dice
Contrast Stretching           Canny      semua       0.1531      0.2128
Contrast Stretching           Canny      uji         0.1705      0.2250
Contrast Stretching           Laplacian  semua       0.2209      0.3314
Contrast Stretching           Laplacian  uji         0.2567      0.3742
Contrast Stretching + Median  Canny      semua       0.2175      0.3078
Contrast Stretching + Median  Canny      uji         0.2282      0.3077
Contrast Stretching + Median  Laplacian  semua       0.2123      0.3218
Contrast Stretching + Median  Laplacian  uji         0.2243      0.3378
```

Interpretasi: Seluruh nilai IoU metode klasik berada di bawah 0,26 pada data uji, dan hanya 6 sampai 13 dari 65 citra uji yang IoU-nya mencapai 0,5. Laplacian unggul pada masukan Contrast Stretching (IoU 0,2567 berbanding 0,1705 untuk Canny), sedangkan pada masukan Contrast Stretching + Median Canny sedikit lebih baik (0,2282 berbanding 0,2243). Karena arah selisihnya berbeda antar masukan dan selisihnya kecil, tidak ada metode klasik yang konsisten lebih baik. Nilai pada seluruh citra (327) tidak jauh berbeda dari data uji, yaitu 0,1531 sampai 0,2209.

5.2.5 Keluaran Canny dan Laplacian sebagai Masukan Deep Learning

Pada alur deep learning, keluaran Metode 1 (Canny) dan Metode 2 (Laplacian) pada resolusi asli diperkecil ke 128 x 128 dengan interpolasi INTER_AREA, lalu dipakai sebagai masukan U-Net dan DeepLabV3. Proses ini dijalankan untuk kedua masukan enhancement di dalam uas_dl.py, dan peta tepi tidak disimpan ke berkas terpisah.

```
S = args.size
DATA = {}  # (masukan enhancement, sumber) -> larik N x S x S
for inp in inputs:
    E = {s: [] for s in SRC}
    for p in files:
        g = CAND[inp](cv2.cvtColor(cv2.imread(p, cv2.IMREAD_COLOR), cv2.COLOR_BGR2GRAY))
        E["Citra"].append(cv2.resize(g, (S, S), interpolation=cv2.INTER_AREA))
        for m, f in METODE.items():  # keluaran Metode 1 / Metode 2 pada resolusi asli,
lalu diperkecil
            E[m].append(cv2.resize(f(g), (S, S), interpolation=cv2.INTER_AREA))
    for s in SRC:
        DATA[(inp, s)] = np.stack(E[s])
    print("%-30s peta tepi: Canny %.2f%%, Laplacian %.2f%% piksel tepi" % (
        inp, 100 * (DATA[(inp, "Canny")] > 0).mean(), 100 * (DATA[(inp, "Laplacian")] >
0).mean()), flush=True)
```

Output:

```
Contrast Stretching            peta tepi: Canny 54.92%, Laplacian 21.47% piksel tepi
Contrast Stretching + Median   peta tepi: Canny 30.24%, Laplacian 20.86% piksel tepi
```

Interpretasi: Terbentuk empat kumpulan peta tepi, masing-masing berisi 327 citra berukuran 128 x 128. Nilai rata-rata piksel tepi dihitung sebagai persentase piksel yang tidak bernol setelah pengecilan citra. Canny pada Contrast Stretching menghasilkan piksel tepi paling banyak (54,92%), turun menjadi 30,24% setelah filter median, sedangkan Laplacian relatif stabil (21,47% dan 20,86%).

5.3 Segmentasi Deep Learning: U-Net dan DeepLabV3

Subbab ini membahas masukan, arsitektur, pengaturan pelatihan, dan hasil U-Net dan DeepLabV3. Sesuai alur pada ketentuan tugas, kedua model dilanjutkan dari keluaran metode klasik: peta tepi hasil Canny dan Laplacian menjadi masukan model, sedangkan mask ground truth menjadi target. Kedua model dilatih dari nol tanpa bobot pretrained pada citra 128 x 128 satu kanal.

5.3.1 Masukan dan Target Model

Untuk setiap masukan enhancement (Contrast Stretching dan Contrast Stretching + Median) dan setiap metode klasik (Canny dan Laplacian), dilatih satu U-Net dan satu DeepLabV3 sehingga terbentuk 2 x 2 x 2 = 8 pelatihan terpisah. Peta tepi dibentuk di dalam uas_dl.py, lalu dinormalisasi ke rentang 0 sampai 1, sedangkan data latih, validasi, dan uji memakai pembagian yang sama seperti metode klasik (262 citra latih, dengan 15% di antaranya sebagai validasi, dan 65 citra uji).

```
d = np.load(args.npz)
Y, test, names = d["Y"].astype(np.float32)[..., None], d["test"], [str(n) for n in
d["names"]]
inputs = [str(d["Xname%d" % k]) for k in range(10) if "Xname%d" % k in d.files]
SRC = ["Citra", "Canny", "Laplacian"]


for inp in inputs:
    for src in SRC:
        X = (DATA[(inp, src)].astype(np.float32) / 255.0)[..., None]   # masukan model
```

Interpretasi: Variabel X pada setiap pelatihan adalah peta tepi Canny atau Laplacian dari masukan yang bersangkutan, sedangkan Y adalah mask ground truth. Dengan cara ini model U-Net dari Canny dan U-Net dari Laplacian benar-benar berbeda karena menerima masukan yang berbeda.

5.3.2 Arsitektur U-Net

U-Net terdiri dari jalur encoder dengan empat tingkat blok konvolusi ganda (jumlah filter 16, 32, 64, dan 128) yang diikuti max pooling, bagian bottleneck dengan 256 filter, dan jalur decoder yang memakai transposed convolution serta skip connection dari encoder. Keluarannya adalah satu kanal dengan aktivasi sigmoid. Setiap konvolusi diikuti batch normalization dan ReLU.

```
def conv(x, f, k=3, rate=1):
    x = L.Conv2D(f, k, padding="same", dilation_rate=rate, use_bias=False)(x)
    return L.ReLU()(L.BatchNormalization()(x))



def unet(S=128, b=16):
    i = keras.Input((S, S, 1)); x = i; sk = []
    for f in (b, 2 * b, 4 * b, 8 * b):
        x = conv(conv(x, f), f); sk.append(x); x = L.MaxPooling2D()(x)
    x = conv(conv(x, 16 * b), 16 * b)
    for f, s in zip((8 * b, 4 * b, 2 * b, b), reversed(sk)):
        x = L.Conv2DTranspose(f, 2, strides=2, padding="same")(x)
        x = conv(conv(L.Concatenate()([x, s]), f), f)
    return keras.Model(i, L.Conv2D(1, 1, activation="sigmoid")(x), name="UNet")
```

Interpretasi: Model U-Net memiliki 1.945.233 parameter (dl_metrik.csv). Skip connection menggabungkan keluaran setiap tingkat encoder dengan tingkat decoder yang sesuai.

5.3.3 Arsitektur DeepLabV3

DeepLabV3 yang dipakai adalah versi ringan dengan encoder ResNet kecil (output stride 8, blok terakhir memakai atrous convolution dengan rate 2), modul ASPP (satu konvolusi 1x1, tiga konvolusi atrous dengan rate 6, 12, dan 18, serta pooling global), dan upsampling bilinear ke ukuran awal. Versi ini bukan DeepLabV3 besar berbobot pretrained.

```
def res block(x, f, stride=1, rate=1):
    s = x
    y = conv(x, f, 3, rate) if stride == 1 else L.Conv2D(f, 3, strides=stride,
padding="same", use_bias=False)(x)
    if stride != 1:
        y = L.ReLU()(L.BatchNormalization()(y))
    y = L.BatchNormalization()(L.Conv2D(f, 3, padding="same", dilation_rate=rate,
use_bias=False)(y))
    if stride != 1 or s.shape[-1] != f:
        s = L.BatchNormalization()(L.Conv2D(f, 1, strides=stride, use_bias=False)(s))
    return L.ReLU()(L.Add()([y, s]))



def deeplabv3(S=128, b=16):
    # encoder ResNet kecil (output stride 8, blok terakhir atrous) + ASPP + upsampling
bilinear
    i = keras.Input((S, S, 1))
    x = conv(i, b)
    x = res_block(x, 2 * b, 2)                  # /2
    x = res_block(x, 4 * b, 2)                  # /4
    x = res_block(x, 8 * b, 2)                  # /8
    x = res_block(x, 16 * b, 1, rate=2)         # atrous, tetap /8
    n = S // 8
    br = [conv(x, 8 * b, 1)] + [conv(x, 8 * b, 3, r) for r in (6, 12, 18)]
    ip = L.GlobalAveragePooling2D(keepdims=True)(x)
    ip = L.UpSampling2D(n, interpolation="bilinear")(conv(ip, 8 * b, 1))
```

```
    x = conv(L.Concatenate()(br + [ip]), 8 * b, 1)
    x = L.Dropout(0.1)(x)
    x = L.UpSampling2D(8, interpolation="bilinear")(L.Conv2D(1, 1)(x))
    return keras.Model(i, L.Activation("sigmoid")(x), name="DeepLabV3")
```

Interpretasi: Model DeepLabV3 memiliki 2.259.921 parameter, sedikit lebih banyak daripada U-Net. Keluaran modul ASPP berada pada resolusi 1/8 dari citra masukan dan diperbesar delapan kali dengan upsampling bilinear.

5.3.4 Loss, Augmentasi, dan Pelatihan

Loss yang dipakai adalah gabungan binary cross-entropy dan Dice loss. Optimizer yang dipakai adalah Adam dengan laju belajar awal 0,001, yang dikurangi setengahnya (ReduceLROnPlateau) jika tidak membaik selama 5 epoch. Augmentasi berupa cermin horizontal, geser 10%, zoom 15%, dan rotasi kecil diterapkan pada peta tepi dan mask secara bersamaan. Bobot dengan val_loss terbaik setelah epoch ke-10 disimpan karena batch normalization belum stabil pada epoch awal.

```
def dice_loss(y, p):
    inter = tf.reduce_sum(y * p, (1, 2, 3))
    return 1 - tf.reduce_mean((2 * inter + 1) / (tf.reduce_sum(y, (1, 2, 3)) +
tf.reduce_sum(p, (1, 2, 3)) + 1))



def loss(y, p):
    return tf.reduce_mean(keras.losses.binary_crossentropy(y, p)) + dice_loss(y, p)

AUG = keras.Sequential([L.RandomFlip("horizontal"), L.RandomTranslation(0.1, 0.1),
L.RandomZoom(0.15), L.RandomRotation(0.05)])



def augment(x, y):
    z = AUG(tf.concat([x, y], -1), training=True)
    return z[..., :1], tf.cast(z[..., 1:] > 0.5, tf.float32)

        for mk in (unet, deeplabv3):
            keras.utils.set_random_seed(42)
            m = mk()
            m.compile(keras.optimizers.Adam(1e-3), loss=loss)
            ds = tf.data.Dataset.from_tensor_slices((X[fit_idx], Y[fit_idx])).shuffle(256,
seed=1).batch(args.batch).map(augment)
            cb = [BestAfter(10), keras.callbacks.ReduceLROnPlateau(patience=5, factor=0.5,
min_lr=1e-5)]
            h = m.fit(ds, validation_data=(X[va_idx], Y[va_idx]), epochs=args.epochs,
callbacks=cb, verbose=0)
```

Interpretasi: Loss pelatihan merupakan penjumlahan binary cross-entropy dan Dice loss, sedangkan evaluasi memakai ambang 0,5 pada keluaran sigmoid. Setiap model dilatih 60 epoch dengan seed tetap 42, dan evaluasi dilakukan pada 65 citra uji yang tidak dipakai selama pelatihan.

5.3.5 Hasil U-Net dan DeepLabV3

Output:

```
input, masukan_model, model, epoch, IoU_uji, Dice_uji, IoU>=0.5, n_uji, parameter
Contrast Stretching, Citra, UNet, 60, 0.815, 0.8939, 64, 65, 1945233
Contrast Stretching, Citra, DeepLabV3, 60, 0.7527, 0.8536, 62, 65, 2259921
Contrast Stretching, Canny, UNet, 60, 0.6995, 0.8067, 57, 65, 1945233
Contrast Stretching, Canny, DeepLabV3, 60, 0.6634, 0.7862, 58, 65, 2259921
Contrast Stretching, Laplacian, UNet, 60, 0.7181, 0.829, 59, 65, 1945233
Contrast Stretching, Laplacian, DeepLabV3, 60, 0.6767, 0.799, 61, 65, 2259921
Contrast Stretching + Median, Citra, UNet, 60, 0.8126, 0.8873, 63, 65, 1945233
Contrast Stretching + Median, Citra, DeepLabV3, 60, 0.7745, 0.8683, 63, 65, 2259921
Contrast Stretching + Median, Canny, UNet, 60, 0.7522, 0.8484, 61, 65, 1945233
Contrast Stretching + Median, Canny, DeepLabV3, 60, 0.7361, 0.8379, 61, 65, 2259921
Contrast Stretching + Median, Laplacian, UNet, 60, 0.7034, 0.8156, 59, 65, 1945233
Contrast Stretching + Median, Laplacian, DeepLabV3, 60, 0.6721, 0.7964, 61, 65,
2259921
```

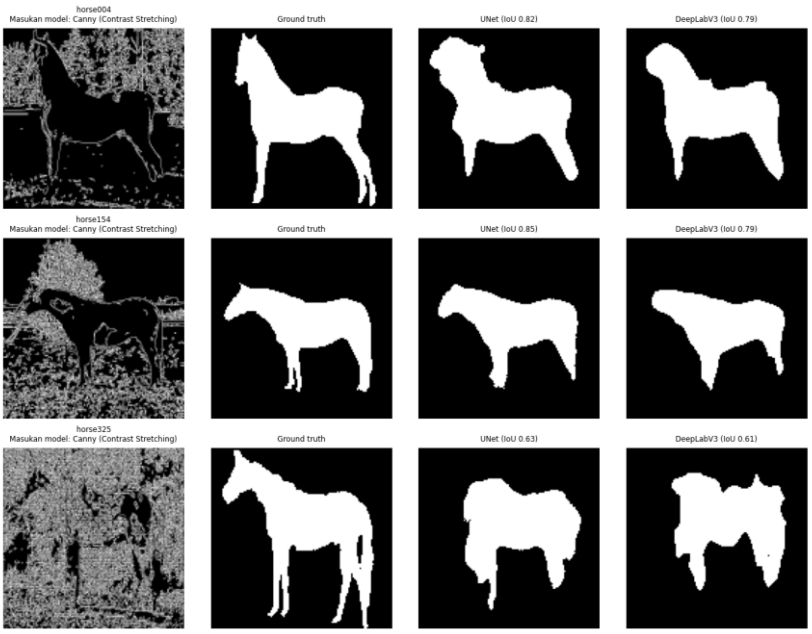
Tabel 5.3. IoU dan Dice deep learning pada 65 citra uji (60 epoch)

Masukan	Metode Klasik	Model	IoU	Dice	Citra IoU ≥ 0,5
Contrast Stretching	Canny	U-Net	0,6995	0,8067	57 dari 65
Contrast Stretching	Canny	DeepLabV3	0,6634	0,7862	58 dari 65
Contrast Stretching	Laplacian	U-Net	0,7181	0,8290	59 dari 65
Contrast Stretching	Laplacian	DeepLabV3	0,6767	0,7990	61 dari 65
Contrast Stretching + Median	**Canny**	**U-Net**	**0,7522**	**0,8484**	**61 dari 65**
Contrast Stretching + Median	Canny	DeepLabV3	0,7361	0,8379	61 dari 65
Contrast Stretching + Median	Laplacian	U-Net	0,7034	0,8156	59 dari 65
Contrast Stretching + Median	Laplacian	DeepLabV3	0,6721	0,7964	61 dari 65

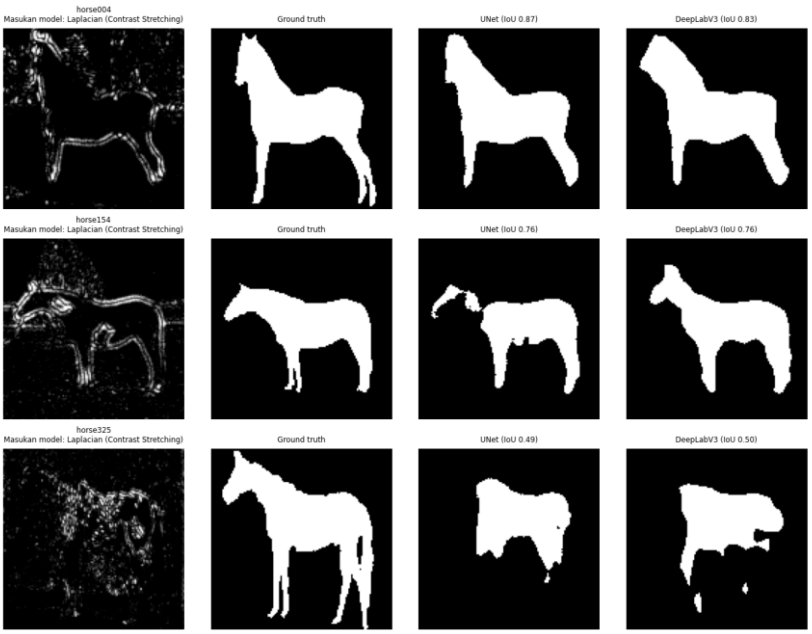
Interpretasi: Empat baris dengan masukan_model Citra adalah pembanding yang dibahas pada Subbab 5.4.3, sedangkan Tabel 5.3 memuat delapan baris sisanya. IoU delapan model berada pada rentang 0,6634 sampai 0,7522. U-Net memiliki IoU lebih tinggi daripada DeepLabV3 pada keempat pasangan masukan dan metode klasik, dengan selisih 0,0361 (Contrast Stretching, Canny), 0,0414 (Contrast Stretching, Laplacian), 0,0161 (Contrast Stretching + Median, Canny), dan 0,0313 (Contrast Stretching + Median, Laplacian). Model terbaik adalah U-Net dengan masukan peta tepi Canny dari Contrast Stretching + Median (IoU 0,7522; Dice 0,8484), dan yang terendah adalah DeepLabV3 dengan masukan peta tepi Canny dari Contrast Stretching (IoU 0,6634).

5.3.6 Contoh Hasil Segmentasi

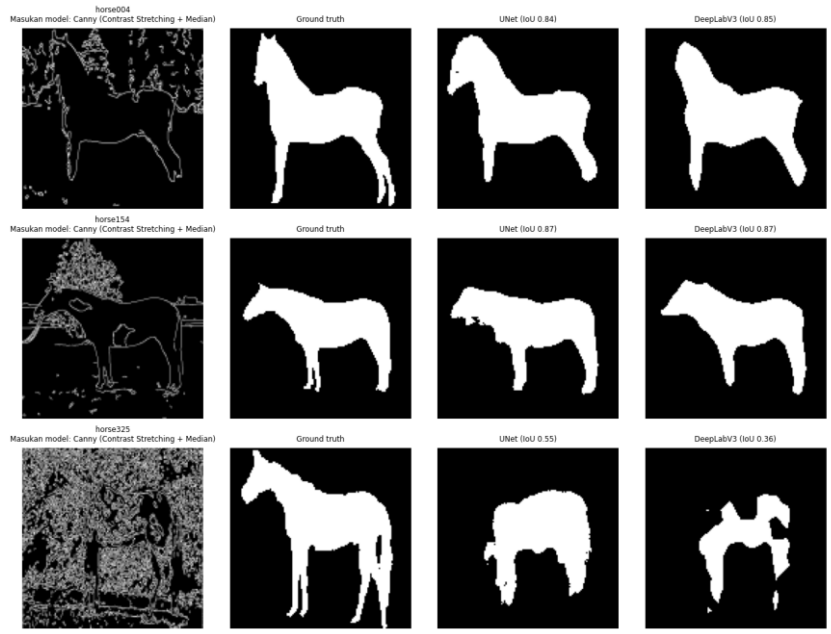
Gambar berikut memperlihatkan tiga citra uji (horse004, horse154, dan horse325) untuk masing-masing kombinasi masukan dan metode klasik, berurutan dari kiri ke kanan: peta tepi yang menjadi masukan model, ground truth, hasil U-Net, dan hasil DeepLabV3.



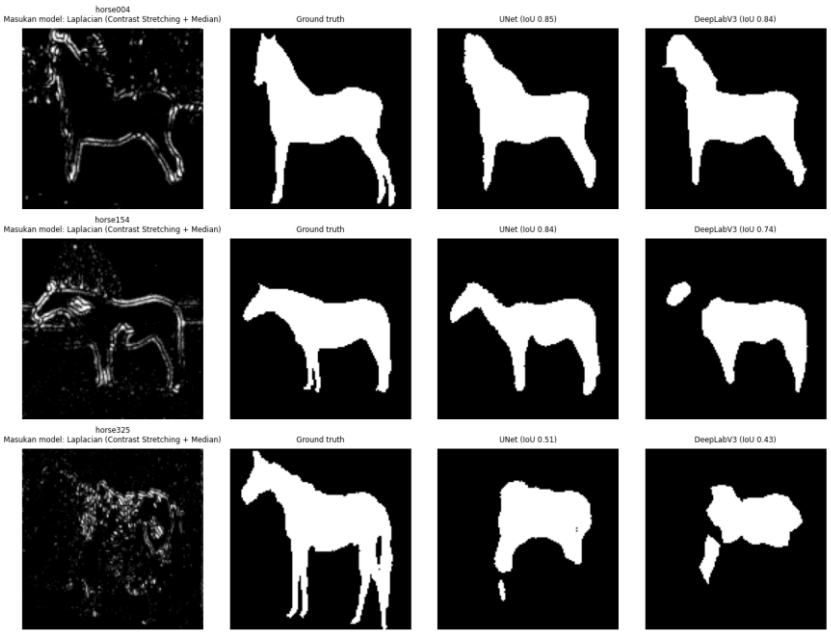
Gambar 5.2. Contoh hasil: Contrast Stretching, keluaran Canny



Gambar 5.3. Contoh hasil: Contrast Stretching, keluaran Laplacian



Gambar 5.4. Contoh hasil: Contrast Stretching + Median, keluaran Canny



Gambar 5.5. Contoh hasil: Contrast Stretching + Median, keluaran Laplacian

Interpretasi: Pada horse004 dan horse154 IoU kedua model berada pada 0,74 sampai 0,87 untuk keempat kombinasi. Pada horse325 (kuda berbelang dengan latar ramai) IoU turun di semua gambar, yaitu 0,63 dan 0,61 (Gambar 5.2), 0,49 dan 0,50 (Gambar 5.3), 0,55 dan 0,36 (Gambar 5.4), serta 0,51 dan 0,43 (Gambar 5.5) untuk U-Net dan DeepLabV3. Peta tepi Canny horse325 pada Gambar 5.2 dan 5.4 dipenuhi tepi dari latar, dan hasil kedua model hanya menyerupai bentuk kuda secara kasar.

5.4 Perbandingan 8 Output Segmentasi

5.4.1 Ringkasan Seluruh Output

Alur pada ketentuan tugas menghasilkan delapan output: untuk setiap masukan enhancement, keluaran Metode 1 (Canny) dan Metode 2 (Laplacian) masing-masing dilanjutkan ke Metode 1 (U-Net) dan Metode 2 (DeepLabV3) deep learning. Nomor output mengikuti diagram: Output 1 sampai 4 untuk masukan Contrast Stretching dan Output 5 sampai 8 untuk masukan Contrast Stretching + Median. Seluruhnya dievaluasi pada 65 citra uji yang sama.

Tabel 5.4. IoU dan Dice kedelapan output pada 65 citra uji

Output	Masukan	Metode Klasik	Metode Deep Learning	IoU	Dice
1	Contrast Stretching	Canny	U-Net	0,6995	0,8067
2	Contrast Stretching	Canny	DeepLabV3	0,6634	0,7862
3	Contrast Stretching	Laplacian	U-Net	0,7181	0,8290
4	Contrast Stretching	Laplacian	DeepLabV3	0,6767	0,7990
5	**Contrast Stretching + Median**	**Canny**	**U-Net**	**0,7522**	**0,8484**
6	Contrast Stretching + Median	Canny	DeepLabV3	0,7361	0,8379
7	Contrast Stretching + Median	Laplacian	U-Net	0,7034	0,8156
8	Contrast Stretching + Median	Laplacian	DeepLabV3	0,6721	0,7964

Interpretasi: Output 5 (Contrast Stretching + Median, Canny, U-Net) memiliki IoU dan Dice tertinggi, yaitu 0,7522 dan 0,8484. Output 2 (Contrast Stretching, Canny, DeepLabV3) memiliki IoU terendah, yaitu 0,6634. Seluruh output memiliki IoU di atas 0,66.

5.4.2 Perbandingan dengan Metode Klasik

Tabel 5.5. IoU metode klasik sendiri dibandingkan dengan lanjutan deep learning (65 citra uji)

Masukan	Metode Klasik	IoU klasik	IoU U-Net	IoU DeepLabV3
Contrast Stretching	Canny	0,1705	0,6995	0,6634
Contrast Stretching	Laplacian	0,2567	0,7181	0,6767
Contrast Stretching + Median	Canny	0,2282	0,7522	0,7361
Contrast Stretching + Median	Laplacian	0,2243	0,7034	0,6721

Interpretasi: Melanjutkan keluaran Canny dan Laplacian ke deep learning menaikkan IoU dari rentang 0,1705 sampai 0,2567 menjadi 0,6634 sampai 0,7522. Peta tepi yang sama, bila

dilanjutkan ke U-Net atau DeepLabV3, menghasilkan mask yang jauh lebih mendekati ground truth daripada aturan sederhana pada metode klasik.

5.4.3 Perbandingan dengan Pelatihan Langsung dari Citra Enhancement

Selain delapan output, uas_dl.py juga melatih U-Net dan DeepLabV3 langsung dari citra hasil enhancement tanpa melalui keluaran Canny atau Laplacian (masukan_model Citra pada dl_metrik.csv). Hasilnya dijadikan pembanding.

Tabel 5.6. IoU pelatihan dari citra enhancement dibandingkan dengan dari peta tepi (65 citra uji)

Masukan	Model	Dari citra	Dari Canny	Dari Laplacian
Contrast Stretching	U-Net	0,8150	0,6995	0,7181
Contrast Stretching	DeepLabV3	0,7527	0,6634	0,6767
Contrast Stretching + Median	U-Net	0,8126	0,7522	0,7034
Contrast Stretching + Median	DeepLabV3	0,7745	0,7361	0,6721

Interpretasi: Pada kedelapan perbandingan, IoU dari peta tepi lebih rendah daripada IoU dari citra enhancement, dengan selisih 0,0384 sampai 0,1155. Hasil ini sejalan dengan dugaan bahwa peta tepi menyimpan informasi yang lebih sedikit daripada citra, tetapi percobaan ini tidak menguji penyebabnya secara langsung. Hasil pada Tabel 5.6 hanya dipakai sebagai pembanding, sedangkan delapan output pada Tabel 5.4 mengikuti alur pada ketentuan tugas.

5.4.4 Pembahasan

U-Net memiliki IoU rata-rata lebih tinggi daripada DeepLabV3 pada keempat pasangan masukan dan metode klasik. Namun jumlah citra uji dengan IoU minimal 0,5 pada DeepLabV3 sama atau lebih banyak daripada U-Net (58 berbanding 57, 61 berbanding 59, 61 berbanding 61, dan 61 berbanding 59), sehingga keunggulan U-Net tampaknya bukan berasal dari jumlah citra yang melewati ambang 0,5, melainkan dari nilai IoU yang lebih tinggi pada citra-citra tersebut.

Pengaruh filter median bergantung pada metode klasik. Pada keluaran Canny, filter median menaikkan IoU U-Net dari 0,6995 menjadi 0,7522 dan IoU DeepLabV3 dari 0,6634 menjadi 0,7361. Pada keluaran Laplacian, IoU sedikit turun, yaitu U-Net dari 0,7181 menjadi 0,7034 dan DeepLabV3 dari 0,6767 menjadi 0,6721. Arah yang sama terlihat pada metode klasik sendiri (Tabel 5.2): IoU Canny naik dari 0,1705 menjadi 0,2282 dan IoU Laplacian turun dari 0,2567 menjadi 0,2243. Pada Subbab 5.2.5 terlihat juga bahwa peta tepi Canny dari citra tanpa

median memuat jauh lebih banyak piksel tepi (54,92%) daripada dengan median (30,24%), sedangkan pada Laplacian jumlahnya hampir sama (21,47% dan 20,86%).

Beberapa catatan diperlukan agar hasil tidak ditafsirkan berlebihan. Pertama, setiap model hanya dilatih satu kali dengan satu pembagian data dan hanya 65 citra uji, sehingga selisih kecil antarmodel, misalnya 0,0161 antara U-Net dan DeepLabV3 pada Output 5 dan 6, belum tentu bermakna. Kedua, model deep learning dievaluasi pada citra 128 x 128, sedangkan Canny dan Laplacian dievaluasi pada resolusi asli, sehingga perbandingan pada Tabel 5.5 bersifat kasar. Ketiga, DeepLabV3 yang dipakai adalah versi ringan tanpa bobot pretrained.

BAB VI
SARAN DAN KESIMPULAN

6.1 Kesimpulan

Tabel 6.1. Ringkasan tahapan percobaan

Tahapan	Metode	Hasil Utama
Data	Weizmann Horse Database	327 citra dan mask; 262 citra latih dan 65 citra uji
Enhancement	6 kandidat (HE, Contrast Stretching, CLAHE, masing-masing tanpa dan dengan median kernel 5)	Contrast Stretching terbaik (PSNR 25,77 dB; MSE 455,56)
Masukan segmentasi	Kandidat terbaik dan kandidat terbaik berfilter	Contrast Stretching dan Contrast Stretching + Median, menghasilkan 8 output
Metode Klasik	Canny dan Laplacian	IoU 0,1705 sampai 0,2567
Metode Deep Learning	U-Net dan DeepLabV3 dari keluaran Canny dan Laplacian, 60 epoch	IoU 0,6634 sampai 0,7522
Terbaik	Output 5: Contrast Stretching + Median, Canny, U-Net	IoU 0,7522; Dice 0,8484

Berdasarkan hasil penelitian, dari enam kandidat praproses, Contrast Stretching tanpa filter memberi PSNR tertinggi (25,77 dB), dan filter median tidak meningkatkan nilai PSNR pada ketiga metode enhancement. Karena kandidat terbaik berasal dari kelompok tanpa filter, masukan segmentasi adalah Contrast Stretching dan Contrast Stretching + Median, dan alur Canny atau Laplacian yang dilanjutkan ke U-Net atau DeepLabV3 menghasilkan delapan output.

Melanjutkan keluaran Canny dan Laplacian ke deep learning menaikkan IoU dari 0,1705 sampai 0,2567 menjadi 0,6634 sampai 0,7522. Output terbaik adalah U-Net dengan masukan keluaran Canny dari Contrast Stretching + Median (IoU 0,7522; Dice 0,8484), dan U-Net lebih tinggi daripada DeepLabV3 pada semua pasangan, dengan catatan selisihnya kecil (0,0161 sampai 0,0414). Sebagai pembanding, pelatihan langsung dari citra enhancement memberi IoU lebih tinggi (0,7527 sampai 0,8150), tetapi tidak mengikuti alur pada ketentuan tugas.

Keterbatasan percobaan ini adalah setiap model hanya dilatih satu kali dengan satu pembagian data, jumlah citra uji hanya 65, DeepLabV3 yang dipakai adalah versi ringan tanpa bobot pretrained, serta resolusi evaluasi yang berbeda antara metode klasik dan deep learning.

6.2 Saran

Berdasarkan hasil penelitian, pengembangan selanjutnya disarankan sebagai berikut:

1. Mengulang pelatihan dengan beberapa pembagian data dan seed yang berbeda, kemudian melaporkan rata-rata dan sebarannya, agar selisih kecil antara U-Net dan DeepLabV3 dapat diuji lebih meyakinkan.
2. Mengevaluasi metode klasik dan deep learning pada resolusi yang sama agar perbandingan keduanya lebih adil.
3. Menguji DeepLabV3 dan U-Net berbobot pretrained, serta variasi ukuran kernel median atau penggunaan Gaussian filter.
4. Menguji masukan model yang menggabungkan peta tepi dengan citra enhancement sebagai dua kanal, mengingat pelatihan langsung dari citra memberi IoU lebih tinggi daripada dari peta tepi saja.
5. Menetapkan aturan agregasi pemilihan kandidat terbaik. Pada penelitian ini kandidat dipilih dari rata-rata PSNR. Jika pemilihan dilakukan dengan voting per citra (varian dengan MSE terkecil pada setiap citra), Contrast Stretching tetap terbaik (244 dari 327 citra), tetapi kandidat berfilter terbaik menjadi CLAHE + Median (48 citra) dan bukan Contrast Stretching + Median (3 citra), sehingga masukan kedua dapat berbeda.

DAFTAR PUSTAKA

Ahmed, S., & Islam, S. (2023). Methods in detection of median filtering in digital images: A survey. Multimedia Tools and Applications, 82(28), 43945–43965. https://doi.org/10.1007/s11042-023-14835-x

Aviantoro, K., & Darnita, Y. (2024). Implementasi Wiener, Contrast Stretching, Sharpening Filter pada citra semangka menggunakan MSE, RMSE, dan PSNR. Djtechno: Jurnal Teknologi Informasi, 5(2), 195–205. https://doi.org/10.46576/djtechno.v5i2.4613

Borenstein, E., & Ullman, S. (2002). Class-specific, top-down segmentation. In Computer Vision – ECCV 2002 (pp. 109–124). Springer. https://doi.org/10.1007/3-540-47967-8_8

Canny, J. (1986). A computational approach to edge detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, PAMI-8(6), 679–698.

Chen, L.-C., Papandreou, G., Schroff, F., & Adam, H. (2017). Rethinking atrous convolution for semantic image segmentation. arXiv:1706.05587. https://arxiv.org/abs/1706.05587

Dijaya, R., & Setiawan, H. (2023). Buku ajar pengolahan citra digital. UMSIDA Press. https://doi.org/10.21070/2023/978-623-464-075-5

Marr, D., & Hildreth, E. (1980). Theory of edge detection. Proceedings of the Royal Society of London. Series B, Biological Sciences, 207(1167), 187–217. https://doi.org/10.1098/rspb.1980.0020

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015 (Lecture Notes in Computer Science, Vol. 9351, pp. 234–241). Springer. https://doi.org/10.1007/978-3-319-24574-4_28

Sharma, R., & Kamra, A. (2023). A review on CLAHE based enhancement techniques. Proceedings of the International Conference on Contemporary Computing and Informatics (IC3I 2023), 321–325. https://doi.org/10.1109/IC3I59117.2023.10397722

LAMPIRAN

Struktur folder proyek (UAS_Kelompok)

```
UAS_Kelompok/
|-- README.md
|-- requirements.txt
|-- uas_tahap1.py        enhancement, evaluasi 6 kandidat,
Canny, Laplacian, ekspor data
|-- uas_dl.py            peta tepi Canny/Laplacian, U-Net,
DeepLabV3 (8 output + 4 pembanding)
|-- weizmann_horse_db/   horse/ (327 citra) dan mask/ (327
mask)
`-- hasil/
    |-- ringkasan_tahap1.txt
    |-- evaluasi_6_kandidat.csv
    |-- segmentasi_klasik.csv
    |-- dl_metrik.csv
    |-- percobaan_30epoch_dl_metrik.csv
    |-- data_dl.npz
    `-- fig_kandidat.png, fig_dl_*.png
```

Link Github

https://github.com/Taraka2027/UAS-COMVIS-Kel-13

Link Dataset

https://www.kaggle.com/datasets/ztaihong/weizmann-horse-database